

변수와 연산자

1. 변수와 데이터 타입

변수는 값을 저장하는 공간의 이름이다. 변수는 프로그램의 가장 기본 요소이다. 변수는 변수명, 주소, 타입, 값으로 구성되는데, PHP는 다른 언어와 달리 변수의 주소를 직접 참조할 수 없어서 변수명, 타입, 값만 이용한다.

```
$a = 1;
```

- 변수는 '\$' 기호로 시작한다.
- 변수명은 문자, 숫자, _(밑줄)로 구성된다.
 - . 첫 글자는 반드시 문자로 시작한다.
 - . 변수명에 한글을 쓸 수 있으나 사용하지 않는다.
- 변수명은 대소문자를 구별한다.
 - . \$abc, \$ABC는 완전히 다른 변수다.

```
$a = 1;  
echo("W$a = {$a}");
```

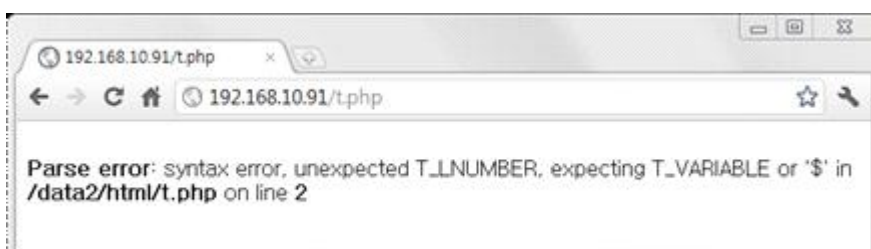
PHP에서 변수는 값이 할당되는 순간 생성된다. c 언어와 같이 미리 선언될 필요가 없다. \$a = 1; 문장이 실행되면 메모리에 \$a의 값을 저장하는 영역이 할당되고 이 영역에 1을 저장한다.

```
$문 = 1;  
echo("W$문 = $문");
```

변수명이 한글로 만들어져도 프로그램에 문제될 것은 없지만 프로그램의 이전이나 백업 등의 작업을 하다보면 원래 문자를 손실할 수도 있다. 그러므로 가능한 변수명에 한글을 사용하는 것은 피하고 있다. 본서에서도 변수명으로 한글을 사용하지 않는다.

```
$2a = 1;  
echo("W$2a = $2a");
```

변수명은 숫자가 먼저 올 수 없다. 문자나 '_' 등의 문자만이 첫머리에 올 수 있다. 변수명에 문제가 있으면 에러가 발생한다.



변수명은 이외에도 공백을 포함하거나 연산자로 쓰이는 '!', @, *, &, %, #, -, +' 와 같은 특수문자는 사용할 수 없다.

보통 변수명에는 특수문자를 사용하지 않지만 사실은 ASCII Code 127~255 사이의 문자는 모두 사용가능하다.

▶ 데이터 타입

PHP는 다음과 같이 다양한 데이터 타입의 변수를 지원한다.

스칼라(scalar) 타입	정수(integer) 실수(float, double) 문자열(string) 불린(boolean)	123, -231, 0x123, 1.23, 1.2E15 "ABC", 'abc', '무궁화' TRUE(1), FALSE(NULL,0)
다중(mixed) 타입	배열(array) 객체(object)	여러 개의 값을 하나의 변수에 저장 속성과 메소드로 구성되는 사용자 정의 데이터 타입

```
$a = 1;
$a = "AA";
echo("W$a = $a");
```

\$a는 1이 저장되었을 때는 정수형 이지만 나중에 "AA"가 저장되면 문자열형으로 자동으로 형이 변환된다. 이런 기능은 프로그래머가 골치 아프게 변수의 타입을 직접 변경시켜줘야 하는 타 언어와 달리 프로그램의 유연성을 제공한다.

```
$bo = TRUE; //ex1.php
$int = (integer)$bo;
$str = (string)$bo;
echo(" W$int = $int<br>
W$str = $str");
```

필요하다면 형을 명시적으로 형 변환(Type casting)할 수 있다. 변수나 수식 앞에 변환하고자 하는 형 명칭을 지정하면 된다.

정수형	(int), (integer)
실수형	(real), (float), (double)
문자열형	(string)
불린형	(bool), (boolean)
이진 문자열형	(binary)
배열형	(array)
객체형	(object)

형을 변환 하는 방법은 이것 이외에도 settype() 함수를 이용하는 방법이 있다.

```
$bo = TRUE;
$int = settype($bo, integer); // $int = (integer)$bo;
$str = settype($bo, string); // $str = (string)$bo;
echo(" W$int = $int<br>
      W$str = $str");
```

PHP에서 함수나 캐스팅 연산자를 이용해 형을 변환하는 과정은 프로그램 내에서 반드시 필요한 과정은 아니다. PHP는 이런 방법을 쓰지 않더라도 아무런 문제없이 자동으로 형을 변경하기 때문이다. 그러나 프로그래밍 과정에 대한 좀 더 정확한 이해를 돕는 측면에서 생각하면 명시적 형 변환 과정은 무척이나 유용한 것임에 틀림없다.

▶ 인용 부호와 문자열

```
$a = "AA";
$b = 'AA';
echo("W$a = $a<br>
      W$b = $b");
```

\$a와 \$b에 저장된 문자열에는 차이가 없다. 변수에 일반 문자를 저장할 때 단일 인용 부호를 사용하건, 이중 인용 부호를 사용하건 차이가 없기 때문이다. 그러나 특수 문자(Escaped Character)가 포함되어 있을 때는 이야기가 달라진다. 특수 문자는 개행(carriage)이나 탭과 같이 대부분 화면을 제어하거나 특별한 기호를 표현하기 위해서 사용되는 것들로 대부분 특별한 용도에 이용된다. 만일 이것을 단일 인용부호 속에 넣게 되면 특수 문자가 아니라 일반 문자로 해석하게 된다.

\n	개행 문자(newline)
\r	개행 문자(carriage)
\t	탭 문자(horizontal tab)
\\	\ 문자
\\$	\$ 문자
\"	" 문자

백슬러시('\')를 표현 하고 싶은 경우 키보드를 찾아보면 백슬러시가 아니라 원화 기호('W')로 나와 있는 경우가 많이 있다. 이것은 OS나 사용 중인 프로그램의 폰트에 따라 달라지는데, 대부분 한글 폰트에서 백슬러시의 코드를 원화 기호로 바꾸어 두었기 때문이다. 이로 인해 본서의 지문에도 백슬러시가 아니라 대부분 원화 기호가 쓰이고 있다. 그러나 가능한 읽을 때 백슬러시라 읽어야 하고, 비록 화면에 원화 기호가 보이더라도 백슬러시라고 이해하길 바란다.

어쨌든 백슬러시는 특수 문자를 시작하는 기호로 쓰인다. 그러나 만일 이를 일반 문자로 사용하고 싶다면 단일 인용 부호를 이용하면 된다.

```
$a = "Wt AA";
$b = 'Wt AA';
echo("W$a = $a<br>
      W$b = $b");
```

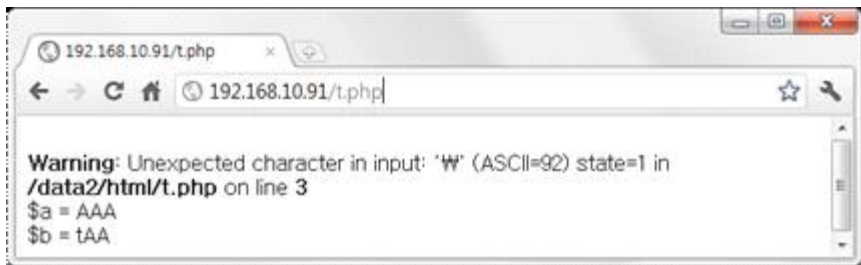
이렇게 동일한 문자열을 각각 단일 인용 부호와 이중 인용 부호로 지정한 다음 내용을 출력하면 \$a는 'AA'라고 출력 되지만 \$b는 'Wt AA'로 출력된다. 좀 더 정확한 내용을 확인하고 싶으면 페이지 소스를 확인하면 좀 더 명확히 확인할 수 있다. 그리고 우리가 결과를 확인하는 화면은 브라우저 화면이므로 대부분의 특수 문자는 무시된다. 이로 인해 'Wn'이나 'Wt'같은 특수 문자들은 사실상 무시된다.

```
$a = TRUE;
$b = "TRUE";
echo("W$a = $a<br>      // 1 출력
      W$b = $b");      // TRUE 출력
```

문자열은 지정하기 위해서는 반드시 인용 부호를 사용해야한다. PHP는 인용 부호 내에 문자열을 일반 문자열로 인식하기 때문이다. 그러므로 인용 부호가 없는 \$a의 TRUE는 문자열이 아니라 참과 거짓을 의미하는 불린형 데이터로 판단하고, \$b의 "TRUE"를 문자열로 판단한다.

```
$a = AAA; // $a = "AAA" 와 동일하다.
$b = WtAA; // W는 단순 실수로 간주하고 경고 메시지를 보내지만 tAA는 유효하다.
echo("W$a = $a<br>
      W$b = $b");
```

만일 문자열을 정의하면서 인용 부호를 생각하면 어찌될까? 지정된 문자열이 TRUE나 AND와 같이 예약어인 경우 예약된 의미로 해석하지만 예약어가 아닌 경우 일반 문자열로 인식한다. 그러나 인용 부호를 생략한 문자열을 사용하는 것은 매우 위험한 짓으로 간주된다. 위의 경우 인용 부호 없이 'W'를 만나면 이를 단순한 오타로 해석하므로 해당 문자는 무시되고 \$b에는 'tAA'라는 문자열이 저장된다. 그리고 경고와 내용을 출력한다.



인용 부호 미지정으로 인한 경고

자동 형 변환과 관련된 다음 프로그램을 실행하고 결과를 확인한다.

자동 형 변환 : 1-1.php		
1 :	<?	
2 :	\$var1 = "1";	
3 :	\$var1++;	// ASCII 49번에서 50번으로 1증가
4 :	\$var2 = "a";	
5 :	\$var2++;	// ASCII 97번에서 98번으로 1증가
6 :	\$var3 = 10 + "11";	// 10 + 11
7 :	\$var4 = 10 + 1.2;	// 10 + 1.2

```

8 :   $var5 = 10 + "1.1e2";    // 10 + 110(실수)
9 :   $var6 = 10 + "A11";      // A11은 해석 못하므로 무시함
10 :  $var7 = 10 + "11A";      // 10 + 11
11 :  echo("W$var1 = {$var1} <br>");
12 :  echo("W$var2 = {$var2} <br>");
13 :  echo("W$var3 = {$var3} <br>");
14 :  echo("W$var4 = {$var4} <br>");
15 :  echo("W$var5 = {$var5} <br>");
16 :  echo("W$var6 = {$var6} <br>");
17 :  echo("W$var7 = {$var7} <br>");
18 :  ?>

```

불린형의 명시적인 형 변환과 관련된 다음 프로그램을 실행하고 결과를 확인한다.

명시적 형 변환 : 1-2.php

```

1 :  <?
2 :   $boo = TRUE;              // $boo는 불린형으로 TRUE를 저장한다.
3 :   $int = (integer)$boo;     // TRUE를 숫자로 바꾼다.
4 :   $str = (string)$boo;      // TRUE를 문자로 저장한다.
5 :   $bt = gettype($boo);      // 데이터 타입을 출력한다.
6 :   $it = gettype($int);
7 :   $st = gettype($str);
8 :   echo("W$boo = {$boo}_{$bt}<br>");
9 :   echo("W$int = {$int}_{$it}<br>");
10 :  echo("W$str = {$str}_{$st}<br><hr>");
11 :
12 :  $boo = FALSE;              // $boo는 불린형으로 FALSE를 저장한다.
13 :  $int = (integer)$boo;      // FALSE를 숫자로 바꾼다.
14 :  $str = (string)$boo;       // FALSE를 문자로 저장한다.
15 :  $bt = gettype($boo);      // 데이터 타입을 출력한다.
16 :  $it = gettype($int);
17 :  $st = gettype($str);
18 :  echo("W$boo = {$boo}_{$bt}<br>");
19 :  echo("W$int = {$int}_{$it}<br>");
20 :  echo("W$str = {$str}_{$st}");
21 :  ?>

```

▶ 산술 연산자

산술연산자는 가장 기본적인 수학 연산자로 더하기, 나누기, 곱하기 등의 연산을 수행한다.

연산자	설명
+	덧셈
-	뺄셈
*	곱셈
/	나눗셈
%	나머지

산술 연산자 예제 프로그램

산술 연산자 : 1-3.php	
1 :	<?
2 :	\$a = 10;
3 :	\$b = 4;
4 :	\$add = \$a + \$b;
5 :	\$sub = \$a - \$b;
6 :	\$mul = \$a * \$b;
7 :	\$div = \$a / \$b;
8 :	\$mod = \$a % \$b;
9 :	echo("W\$a + W\$b는 {\$add}입니다. ");
10 :	echo("W\$a - W\$b는 {\$sub}입니다. ");
11 :	echo("W\$a * W\$b는 {\$mul}입니다. ");
12 :	echo("W\$a / W\$b는 {\$div}입니다. ");
13 :	echo("W\$a % W\$b는 {\$mod}입니다. ");
14 :	?>

▶ 대입 연산자

대입 연산자는 우변의 값을 좌변에 대입하는 연산자를 의미한다. 보통 '='을 많이 이용하지만 PHP는 산술 연산자를 이용한 더 다양한 대입연산자를 제공한다.

연산자	설명	산술 연산자와의 관계
=	우변의 값을 좌변에 대입한다.	\$a = \$b
+=	우, 좌변의 더한 값을 좌변에 대입한다.	\$a += \$b → \$a = \$a + \$b
-=	우변에서 좌변을 뺀 값을 좌변에 대입한다.	\$a -= \$b → \$a = \$a - \$b
*=	우, 좌변의 곱한 값을 좌변에 대입한다.	\$a *= \$b → \$a = \$a * \$b
/=	우변을 좌변으로 나눈 값을 좌변에 대입한다.	\$a /= \$b → \$a = \$a / \$b
%=	우변을 좌변으로 나누어 나머지 값을 좌변에 대입한다.	\$a %= \$b → \$a = \$a % \$b

대입 연산자 예제 프로그램

대입연산자 : 1-4.php	
1 :	<?
2 :	\$a = 10;
3 :	\$b = 4;
4 :	\$a += \$b; // \$a = \$a + \$b
5 :	echo("W\$a += W\$b의 결과는 {\$a}입니다. ");
6 :	\$a -= \$b; // \$a = \$a - \$b
7 :	echo("W\$a -= W\$b의 결과는 {\$a}입니다. ");
8 :	\$a *= \$b; // \$a = \$a * \$b
9 :	echo("W\$a *= W\$b의 결과는 {\$a}입니다. ");
10 :	\$a /= \$b; // \$a = \$a / \$b
11 :	echo("W\$a /= W\$b의 결과는 {\$a}입니다. ");
12 :	\$a %= \$b; // \$a = \$a % \$b
13 :	echo("W\$a %= W\$b의 결과는 {\$a}입니다. ");
14 :	?>

▶ 증감 연산자

프로그램에서는 1증가하거나 1감소하는 순환문을 매우 많이 사용한다. 이들은 대입 연산자나 산술 연산자로 표현하기도 하지만 이런 특별한 연산을 위한 전용 연산자인 증감 연산자를 이용할 수도 있다.

연산자	설명
\$a++	변수를 사용하고 값을 1 증가시킨다.
++\$a	값을 1 증가 시킨 후 변수를 사용한다.
\$a--	변수를 사용하고 값을 1 감소시킨다.
--\$a	값을 1 감소 시킨 후 변수를 사용한다.

증감 연산자	산술 연산자	증감 연산자	산술 연산자
\$b = ++\$a	\$a = \$a + 1 \$b = \$a	\$b = --\$a	\$a = \$a - 1 \$b = \$a
\$b = \$a++	\$b = \$a \$a = \$a + 1	\$b = \$a--	\$b = \$a \$a = \$a - 1

증감 연산자와 산술 연산자

▶ 관계 연산자

관계 연산자는 두 개의 값을 비교하는 연산자이다. PHP는 [표 6.8]과 같이 다양한 비교 연산자를 제공한다.

연산자	설명
==	좌변과 우변의 값이 같으면 TRUE
===	좌변과 우변의 값과 데이터 타입이 모두 같으면 TRUE
!=	좌변과 우변의 값이 다르면 TRUE
!==	좌변과 우변의 값과 데이터 타입이 모두 다르면 TRUE

>	좌변의 값이 우변 값 보다 크면 TRUE
<	좌변의 값이 우변 값 보다 작으면 TRUE
>=	좌변의 값이 우변 값 보다 크거나 같으면 TRUE
<=	좌변의 값이 우변 값 보다 작거나 같으면 TRUE

► 논리 연산자

논리 연산은 피 연산자의 논리적인 연산을 수행하는 것으로 보통 비교 연산자와 같이 쓰인다. '!' (논리 부정)을 제외하고 모두 좌변과 우변의 논리 값을 연산한다.

연산자	사용예	설명
&&	\$a && \$b	\$a와 \$b가 모두 TRUE이면 TRUE
	\$a \$b	\$a와 \$b중 하나 이상이 TRUE이면 TRUE
xor	\$a xor \$b	\$a,\$b 논리 값이 서로 다르면 TRUE
!	!\$a	\$a가 TRUE이면 FALSE, \$a가 FALSE이면 TRUE
and	\$a AND \$b	&&와 결과는 같으나 연산자 우선순위가 낮다.
or	\$a or \$b	와 결과는 같으나 연산자 우선순위가 낮다.

```
$a = TRUE;
$b = FALSE;
$c = $a && $b;    // $c = FALSE
$d = $a and $b;    // $d = TRUE
echo("W$a && W$b 는 {$c}입니다.<br>");
echo("W$a and W$b 는 {$d}입니다.<br>");
```

► 문자열 연산자

연산자	사용예	설명
.	\$c = \$a.\$b	두 개의 문자열을 연결한다.(연결 연산자)
.=	\$c .= 'ing'	문자열을 추가한다.

```
$a = "aa"."bb";
$b = 1 . 2;
$c = 1.2;
$d = aa.1;    // Parse error: syntax error
echo("W$a는 {$a}입니다.<br>");    // aabb 출력
echo("W$b는 {$b}입니다.<br>");    // 12 출력
echo("W$c는 {$c}입니다.<br>");    // 1.2 출력 , 소숫점으로 인식된다.
echo("W$d는 {$d}입니다.<br>");    // 에러
```

연결 연산자는 숫자의 소수점과 동일한 문자(".")를 사용함으로 숫자 타입의 데이터와 함께 쓸 때는 주의해야 한다. 숫자 타입의 데이터와 '.'을 사용하는 경우 둘 사이에 공백이 없으면 항상 소숫점으로 인식하기 때문에 '\$d = aa.1'은 문법상의 오류로 인식된다. 이를 '\$d = "aa" . 1'과 같이 쓰면 정상적인 연결 연산자로 인식된다. 연결 연산자로 연산되는 좌변과 우변이 문자라면 인용부호 사용여부에 관계없이 연산 과정에 문제가 없지

만 양쪽에 숫자가 하나라도 있다면 반드시 숫자형 데이터 쪽에는 공백이 필요하다.
 문자 연결을 위한 연결 연산자를 이용한다면 좌변과 우변에 변수를 쓰거나 인용부호를 사용하자. 비록 에러를 발생하지 않지만 '\$d = "aa" . 1'이런 표현보다는 '\$d = "aa"."1"' 이와 같이 표현하는 것이 올바른 표현이다.

```
$a = "AA";
$a .= "BB";
echo("W$a는 {$a}입니다.<br>"); // AABBA출력
```

'.' 문자열을 대상으로 한 대입 연산자이다. 그러므로 '\$a .= \$b'는 '\$a = \$a . \$b' 와 동일하다.

▶ 연산자 우선순위

연산자를 두 개 이상 사용하는 경우 연산자 우선순위에 따라 연산 순서가 결정된다. 앞서 'and' 연산자의 예처럼 이들 순서에 대해 잘 모르는 상태에서 코딩하게 되면 전혀 의외의 결과가 나오기도 한다. [표 6.11]은 이들 연산자에 대한 우선 순위이다.

연산자 우선순위	연산자
1	++, --
2	!
3	*, /, %
4	+, -, .
5	<, <=, >, >=
6	==, ===, !=, !==
7	&&
8	
9	=, +=, -=, *=, /=, %=, .=
10	and
11	xor
12	or

연산자를 두 개 이상 중복해서 사용하는 경우 연산 과정이 연산자 우선순위에 따라 이뤄지도록 방지해서는 안되며 반드시 괄호를 이용 연산 과정을 통제해야 한다.

예를 들어 '\$a = \$b -= ++\$c' 와 같은 코드를 작성했다면 수학적인 결과가 원하는 데로 출력된다하더라도 추후에 있을 유지 보수 과정에 심각한 악영향을 끼칠 수밖에 없다. 그러므로 절대 이런 코드가 프로그램 내에 존재하지 않도록 주의를 기울여야한다.

2. 연산자를 이용한 다양한 프로그램

피제수와 제수를 입력 받아 몫과 나머지를 출력하는 프로그램을 작성한다.

몫과 나머지 구하기 폼 파일 : 1-5.html

```
1 : <html>
2 : <head>
3 : <title>1-5.html 입력 폼파일</title>
4 : <meta http-equiv="content-type" content="text/html; charset=utf-8">
5 : </head>
6 : <form method="post" action="1-5.php">
7 :     피제수 : <input type="text" name="a"><br>
8 :     제수 : <input type="text" name="b"><br>
9 :         <input type="submit" name="확인" value="확인">
10 :        <input type="reset" name="취소" value="취소"><br>
11 : </form>
12 : </html>
```

몫과 나머지 구하기 프로그램 파일 : 1-5.php

```
1 : <?
2 :     $a = $_POST["a"];
3 :     $b = $_POST["b"];
4 :     $mod = $a % $b;           // 나머지 구하기
5 :     $div = ($a - $mod) / $b;  // 몫 구하기
6 :     echo("{ $a }를 { $b }로 나눈 몫은 { $div }이고<br> 나머지는 { $mod }이다.");
7 : ?>
```